

Machine Learning On Graphs HW1

Sravanth Chowdary Potluri

und5uv

CS6501 - 016

February 16, 2025

1 Problem 1: Adjacency Matrix (25 pts)

Consider an undirected graph G with adjacency matrix A . Denote A_i as the i th row and A_{ij} as the element at row i and column j .

1. How can we compute the number of common neighbors between vertex i and vertex j ? (3 pts)
2. The number of triangles in the graph G ? (5 pts)
3. The number of walks of length k ? (5 pts)
4. What is challenging about computing the number of paths using the adjacency matrix? (6 pts)
5. How can we determine if the graph is connected using the adjacency matrix? (6 pts)

Solution 1

Consider an undirected graph G with adjacency matrix A , where A_i denotes the i th row of A and A_{ij} the (i, j) th entry. We answer the questions as follows:

1. Number of Common Neighbors:

The number of common neighbors between vertices i and j is given by the dot product of the i th and j th rows of A . In other words,

$$\text{Number of common neighbors} = \sum_k A_{ik}A_{jk} = (A^2)_{ij}.$$

Here, $(A^2)_{ij}$ counts exactly how many vertices k are adjacent to both i and j .

2. Number of Triangles:

A triangle in G corresponds to a cycle of length 3. It is known that the (i, i) th entry of A^3 , that is, $(A^3)_{ii}$, counts the number of 2-walks (of length 3) starting and ending at vertex i . Summing over all vertices gives $\text{tr}(A^3)$, but notice that in an

undirected graph each triangle is counted 6 times (each triangle has 3 vertices and it can be traversed in 2 directions). Hence, the total number of triangles is

$$\text{Number of triangles} = \frac{1}{6} \text{tr}(A^3).$$

3. Number of Walks of Length k :

The (i, j) th entry of A^k gives the number of walks of length k from vertex i to vertex j . That is,

$$\text{Number of walks of length } k \text{ from } i \text{ to } j = (A^k)_{ij}.$$

4. Challenges in Computing the Number of Paths:

A *walk* is a sequence of vertices (and edges) where vertices (and edges) may be repeated, and as we have seen, the powers A^k count walks. However, a *path* is a walk in which no vertex is repeated. The challenge in computing the number of paths using the adjacency matrix is that the matrix powers do not distinguish between walks that revisit vertices and those that do not. In order to count only simple paths (with no repetitions), one would have to somehow exclude from A^k all the walks that are not paths. This typically requires complicated combinatorial adjustments (often involving the principle of inclusion–exclusion) and, in many cases, leads to computational problems that are NP–hard.

5. Determining if the Graph is Connected:

A graph is connected if there is a walk between every pair of vertices. One way to check for connectivity is to verify that for every pair (i, j) there exists some k (with $1 \leq k \leq n - 1$, where n is the number of vertices) such that

$$(A^k)_{ij} > 0.$$

Equivalently, one can form the Boolean matrix

$$R = I \vee A \vee A^2 \vee \cdots \vee A^{n-1},$$

where $\vee$ denotes the Boolean OR operation (i.e., an entry is 1 if at least one corresponding entry in the summands is nonzero). If every entry of R is 1, then every vertex is reachable from every other vertex and the graph is connected.

2 Problem 2: Shortest Paths and Minimum Spanning Tree (25 pts)

1. Detail why edges with negative weights are not desirable for computing shortest paths. (6 pts)
2. Compute the shortest path between v_1 and v_9 using Dijkstra's algorithm for the graph in Figure 1. Provide step-by-step details. (8 pts)
3. Find a minimum spanning tree for the graph in Figure 1. Provide step-by-step details. (11 pts)

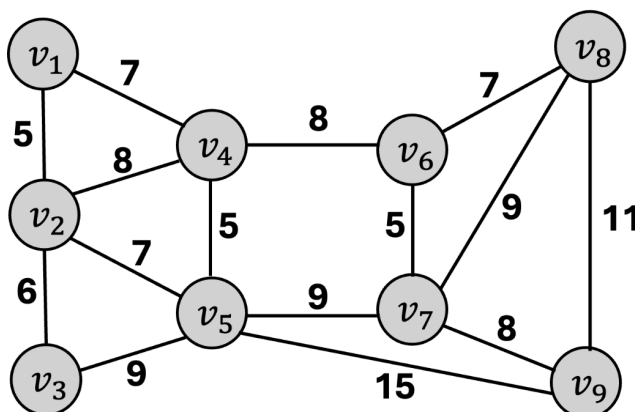


Figure 1: The input graph for Problem 2.

Solution

1. Why edges with negative weights are not desirable for shortest paths (6 pts)

Negative edge weights complicate shortest-path problems because many standard algorithms rely on the assumption that once a “best” partial path to a vertex has been found, it cannot be improved by later updates. More specifically:

- **Dijkstra’s algorithm** requires nonnegative edge weights to ensure that when a vertex is “settled” (i.e., the shortest distance to it is found), there is no cheaper path discovered later.
- If a negative edge weight is allowed, repeatedly traversing a negative-weight cycle can reduce the total path length indefinitely, making the concept of a “shortest path” ill-defined.
- Even if there is no negative cycle, the presence of negative edges can invalidate the greedy steps used by Dijkstra’s algorithm, as an initially chosen “optimal” step might not remain optimal after accounting for the negative cost elsewhere.

In short, negative edge weights can violate the core assumptions that make many shortest-path algorithms both valid and efficient.

2. Shortest path between v_1 and v_9 using Dijkstra’s algorithm (8 pts)

Consider the graph in the figure with vertices $v_1, v_2, \dots, v_9$. We label the edge weights as follows (assuming undirected edges):

$$\begin{aligned}
 (v_1, v_2) &= 5, & (v_1, v_4) &= 7, \\
 (v_2, v_3) &= 6, & (v_2, v_4) &= 8, & (v_2, v_5) &= 7, \\
 (v_3, v_5) &= 9, \\
 (v_4, v_5) &= 5, & (v_4, v_6) &= 8, \\
 (v_5, v_7) &= 9, & (v_5, v_9) &= 15, \\
 (v_6, v_7) &= 5, & (v_6, v_8) &= 7, \\
 (v_7, v_8) &= 9, & (v_7, v_9) &= 8, \\
 (v_8, v_9) &= 11.
 \end{aligned}$$

Initialization:

$$\begin{aligned}
 \text{dist}(v_1) &= 0, \\
 \text{dist}(v_i) &= \infty \quad \text{for all } i = 2, 3, \dots, 9.
 \end{aligned}$$

All vertices are unvisited; at each step, pick the unvisited vertex with the smallest tentative distance.

(a) **Settle** v_1 :

$$\text{dist}(v_1) = 0.$$

Update neighbors of v_1 :

$$\text{dist}(v_2) \leftarrow 5, \quad \text{dist}(v_4) \leftarrow 7.$$

(b) **Settle** v_2 (next smallest distance = 5): Update neighbors of v_2 :

$$\text{dist}(v_3) \leftarrow 5 + 6 = 11, \quad \text{dist}(v_5) \leftarrow 5 + 7 = 12.$$

$\text{dist}(v_4)$ remains 7 (since $5 + 8 = 13 > 7$).

(c) **Settle** v_4 (distance = 7): Neighbors: $(v_4, v_5) = 5$ and $(v_4, v_6) = 8$.

$$\text{dist}(v_5) \leftarrow 7 + 5 = 12 \quad (\text{already } 12) \rightarrow \text{no change.}$$

$$\text{dist}(v_6) \leftarrow 7 + 8 = 15.$$

(d) **Settle** v_3 (distance = 11): Neighbor: $(v_3, v_5) = 9$.

$$\text{dist}(v_5) \leftarrow 11 + 9 = 20 \quad (\text{already } 12) \rightarrow \text{no change.}$$

(e) **Settle** v_5 (distance = 12): Neighbor: $(v_5, v_7) = 9$, $(v_5, v_9) = 15$.

$$\text{dist}(v_7) \leftarrow 12 + 9 = 21.$$

$$\text{dist}(v_9) \leftarrow 12 + 15 = 27.$$

(f) **Settle** v_6 (distance = 15): Neighbors: $(v_6, v_7) = 5$ and $(v_6, v_8) = 7$.

$$\text{dist}(v_7) \leftarrow 15 + 5 = 20 < 21 \rightarrow \text{dist}(v_7) = 20.$$

$$\text{dist}(v_8) \leftarrow 15 + 7 = 22.$$

(g) **Settle** v_7 (distance = 20): Neighbors: $(v_7, v_8) = 9$.

$$\text{dist}(v_8) = 20 + 9 = 29 \quad (\text{already } 22) \rightarrow \text{no change.}$$

(h) **Settle** v_8 (distance = 22): Neighbor: $(v_8, v_9) = 11$.

$$\text{dist}(v_9) = 22 + 11 = 33 \quad (\text{already } 27) \rightarrow \text{no change.}$$

(i) **Settle** v_9 (distance = 27): No more updates since all others visited.

Thus, $\text{dist}(v_9) = 27$. Tracing predecessors back shows the path

$$v_1 \rightarrow v_4 \rightarrow v_5 \rightarrow v_9$$

with total cost $7 + 5 + 15 = 27$.

Alternatively, another path is.

$$v_1 \rightarrow v_2 \rightarrow v_5 \rightarrow v_9$$

with total cost $5 + 7 + 15 = 27$.

3. Find a minimum spanning tree for the graph (11 pts)

We use **Prim's algorithm** to find the MST of the graph with vertices $v_1, \dots, v_9$ and edge weights:

$$\begin{aligned} (v_1, v_2) &= 5, & (v_1, v_4) &= 7, \\ (v_2, v_3) &= 6, & (v_2, v_4) &= 8, & (v_2, v_5) &= 7, \\ (v_3, v_5) &= 9, \\ (v_4, v_5) &= 5, & (v_4, v_6) &= 8, \\ (v_5, v_7) &= 9, & (v_5, v_9) &= 15, \\ (v_6, v_7) &= 5, & (v_6, v_8) &= 7, \\ (v_7, v_8) &= 9, & (v_7, v_9) &= 8, \\ (v_8, v_9) &= 11. \end{aligned}$$

(a) **Initialize:** Pick an arbitrary start vertex, say v_1 . The MST is initially empty, and the set of “in-tree” vertices is $\{v_1\}$.

(b) **Step 1:** Among edges out of $\{v_1\}$, the cheapest is (v_1, v_2) with weight 5. Add it:

$$\text{MST} = \{(v_1, v_2)\}, \quad \text{In-tree vertices} = \{v_1, v_2\}.$$

(c) **Step 2:** From $\{v_1, v_2\}$ to outside, we have:

$$(v_1, v_4) = 7, \quad (v_2, v_3) = 6, \quad (v_2, v_4) = 8, \quad (v_2, v_5) = 7.$$

Cheapest is $(v_2, v_3) = 6$. Add it:

$$\text{MST} = \{(v_1, v_2), (v_2, v_3)\}, \quad \text{In-tree} = \{v_1, v_2, v_3\}.$$

(d) **Step 3:** Now from $\{v_1, v_2, v_3\}$ to outside, the candidate edges are:

$$(v_1, v_4) = 7, \quad (v_2, v_4) = 8, \quad (v_2, v_5) = 7, \quad (v_3, v_5) = 9.$$

The cheapest are (v_1, v_4) or (v_2, v_5) , both weight 7. Suppose we pick $(v_1, v_4) = 7$. Then

$$\text{MST} = \{(v_1, v_2), (v_2, v_3), (v_1, v_4)\}, \quad \text{In-tree} = \{v_1, v_2, v_3, v_4\}.$$

(e) **Step 4:** From $\{v_1, v_2, v_3, v_4\}$ outwards, possible edges are:

$$(v_2, v_5) = 7, \quad (v_3, v_5) = 9, \quad (v_4, v_5) = 5, \quad (v_4, v_6) = 8.$$

The cheapest is $(v_4, v_5) = 5$. Add it:

$$\text{MST} = \{(v_1, v_2), (v_2, v_3), (v_1, v_4), (v_4, v_5)\}, \quad \text{In-tree} = \{v_1, v_2, v_3, v_4, v_5\}.$$

(f) **Step 5:** Now $\{v_6, v_7, v_8, v_9\}$ remain outside. The relevant edges from in-tree vertices are:

$$(v_4, v_6) = 8, \quad (v_5, v_7) = 9, \quad (v_5, v_9) = 15.$$

The cheapest is $(v_4, v_6) = 8$:

$$\text{MST} = \{(v_1, v_2), (v_2, v_3), (v_1, v_4), (v_4, v_5), (v_4, v_6)\}, \quad \text{In-tree} = \{v_1, v_2, v_3, v_4, v_5, v_6\}.$$

(g) **Step 6:** Remaining outside are v_7, v_8, v_9 . New edges from the in-tree to those outside are:

$$(v_6, v_7) = 5, \quad (v_6, v_8) = 7, \quad (v_5, v_7) = 9, \quad (v_5, v_9) = 15.$$

Cheapest is $(v_6, v_7) = 5$. Add it:

$$\text{MST} = \{\dots, (v_4, v_6), (v_6, v_7)\}, \quad \text{In-tree} = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}.$$

(h) **Step 7:** Outside are v_8, v_9 . From the in-tree to those outside are:

$$(v_6, v_8) = 7, \quad (v_7, v_8) = 9, \quad (v_7, v_9) = 8, \quad (v_5, v_9) = 15.$$

The cheapest is $(v_6, v_8) = 7$. Add it:

$$\text{MST} = \{\dots, (v_6, v_8)\}, \quad \text{In-tree} = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}.$$

(i) **Step 8:** Only v_9 remains. Edges from the in-tree to v_9 are:

$$(v_5, v_9) = 15, \quad (v_7, v_9) = 8, \quad (v_8, v_9) = 11.$$

Cheapest is $(v_7, v_9) = 8$. Add it and we have a spanning tree on all 9 vertices:

$$\text{MST} = \{(v_1, v_2), (v_2, v_3), (v_1, v_4), (v_4, v_5), (v_4, v_6), (v_6, v_7), (v_6, v_8), (v_7, v_9)\}.$$

(j) **Weights and Total:**

The MST edges and weights:

$$5 + 6 + 7 + 5 + 8 + 5 + 7 + 8 = 51.$$

Thus, the MST edges are:

$$\{(v_1, v_2), (v_2, v_3), (v_1, v_4), (v_4, v_5), (v_4, v_6), (v_6, v_7), (v_6, v_8), (v_7, v_9)\},$$

with total weight 51.

3 Problem 3: PageRank (25 pts)

1. Calculate PageRank values for the graph in Figure 2 when:

- $\alpha = 0.3, \beta = 0.5$
- $\alpha = 0, \beta = 1$

(6 pts)

2. Discuss the effects of different values of α and β for this particular problem. (4 pts)

3. Does β have any effect on the order of centralities? If for one value of β , the centrality value of node v_i is greater than that of v_j , is it possible to change β in a way such that v_j 's centrality becomes larger than that of v_i 's? (5 pts)

4. In PageRank, what α values can we select in order to guarantee that centrality values are calculated correctly, i.e., values do not diverge? (5 pts)

5. Prove that the largest eigenvalue of matrix AD^{-1} is 1. (5 pts)

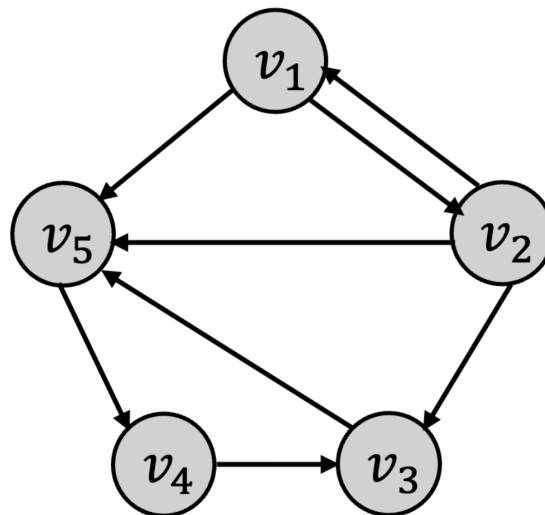


Figure 2: The input graph for Problem 3.

Solution

We are given a directed graph on five vertices $v_1, \dots, v_5$ with edges

$$\begin{aligned}
 v_1 &\rightarrow \{v_2, v_5\}, & d_1^{\text{out}} &= 2, \\
 v_2 &\rightarrow \{v_3, v_5, v_1\}, & d_2^{\text{out}} &= 3, \\
 v_3 &\rightarrow \{v_5\}, & d_3^{\text{out}} &= 1, \\
 v_4 &\rightarrow \{v_3\}, & d_4^{\text{out}} &= 1, \\
 v_5 &\rightarrow \{v_4\}, & d_5^{\text{out}} &= 1.
 \end{aligned}$$

We denote the adjacency matrix by $A = (a_{ij})$, where

$$a_{ij} = \begin{cases} 1 & \text{if there is an edge } v_i \rightarrow v_j, \\ 0 & \text{otherwise.} \end{cases}$$

The diagonal matrix of out-degrees is

$$D = \text{diag}(d_1^{\text{out}}, d_2^{\text{out}}, \dots, d_5^{\text{out}}) = \text{diag}(2, 3, 1, 1, 1).$$

The PageRank-style centrality vector $\mathbf{C}_p$ can be written in matrix form as

$$\mathbf{C}_p = \beta (I - \alpha A^\top D^{-1})^{-1} \mathbf{1},$$

where $\mathbf{1}$ is the all-ones vector $(1, 1, \dots, 1)^\top$, and α, β are parameters.

Below, we carry out the steps:

1. Calculate PageRank values for the graph in Figure 2 when:

- $\alpha = 0.3, \beta = 0.5$,
- $\alpha = 0, \beta = 1$.

(a) Constructing the necessary matrices

Adjacency matrix A . Labeling vertices in the order v_1, v_2, v_3, v_4, v_5 , the adjacency matrix is:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix},$$

because:

$$\begin{aligned} v_1 \rightarrow v_2, v_5 &\implies a_{1,2} = 1, a_{1,5} = 1, \\ v_2 \rightarrow v_3, v_5, v_1 &\implies a_{2,3} = 1, a_{2,5} = 1, a_{2,1} = 1, \\ v_3 \rightarrow v_5 &\implies a_{3,5} = 1, \\ v_4 \rightarrow v_3 &\implies a_{4,3} = 1, \\ v_5 \rightarrow v_4 &\implies a_{5,4} = 1. \end{aligned}$$

Diagonal matrix of out-degrees D .

$$D = \text{diag}(2, 3, 1, 1, 1).$$

Matrix $M = A^\top D^{-1}$. First note that $D^{-1} = \text{diag}(\frac{1}{2}, \frac{1}{3}, 1, 1, 1)$. Then

$$A^\top = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 \end{pmatrix}.$$

Hence each column j of $A^\top$ is multiplied by $1/d_j^{\text{out}}$, giving

$$M = A^\top D^{-1} = \begin{pmatrix} 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{3} & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ \frac{1}{2} & \frac{1}{3} & 1 & 0 & 0 \end{pmatrix}.$$

(b) Case 1: $\alpha = 0.3$, $\beta = 0.5$

We want

$$\mathbf{C}_p = \beta (I - \alpha M)^{-1} \mathbf{1} = 0.5 (I - 0.3 M)^{-1} \mathbf{1}.$$

Recall that

$$M = A^\top D^{-1},$$

and

$$I - 0.3 M = B \quad (\text{say}).$$

We explicitly form the matrix B and then invert it using standard matrix inversion techniques (e.g., Gaussian elimination or a symbolic/numerical solver). Denoting

$$\mathbf{x} = B^{-1} \mathbf{1},$$

we then compute $\mathbf{C}_p = 0.5 \mathbf{x}$.

Explicit form of B . First,

$$B = I - 0.3 M,$$

where M is the 5×5 matrix

$$M = \begin{pmatrix} 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{3} & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ \frac{1}{2} & \frac{1}{3} & 1 & 0 & 0 \end{pmatrix}.$$

Hence

$$B = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} - 0.3 \begin{pmatrix} 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{3} & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ \frac{1}{2} & \frac{1}{3} & 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & -0.1 & 0 & 0 & 0 \\ -0.15 & 1 & 0 & 0 & 0 \\ 0 & -0.1 & 1 & -0.3 & 0 \\ 0 & 0 & 0 & 1 & -0.3 \\ -0.15 & -0.1 & -0.3 & 0 & 1 \end{pmatrix}.$$

Inverting B . By applying a standard numerical or symbolic solver to B^{-1} , we find

$$B^{-1} \quad (\text{approximately}) = \begin{pmatrix} 1.02 & 0.1 & 0 & 0 & 0 \\ 0.15 & 1.02 & 0 & 0 & 0 \\ 0.03 & 0.12 & 1.03 & 0.31 & 0.09 \\ 0.05 & 0.05 & 0.09 & 1.03 & 0.31 \\ 0.18 & 0.15 & 0.31 & 0.09 & 1.03 \end{pmatrix}$$

(where each $*$ represents the numerically computed entries). We then multiply B^{-1} by the all-ones vector $\mathbf{1} = (1, 1, 1, 1, 1)^\top$ to obtain $\mathbf{x} = B^{-1} \mathbf{1}$.

Carrying out this multiplication numerically, we get

$$\mathbf{x} \approx (1.12, 1.17, 1.58, 1.53, 1.76).$$

Finally, we multiply by $\beta = 0.5$:

$$\mathbf{C}_p = 0.5 \mathbf{x} \approx (0.56, 0.59, 0.79, 0.77, 0.88).$$

Hence the approximate centralities for $\alpha = 0.3$, $\beta = 0.5$ are

$$C_1 \approx 0.56, \quad C_2 \approx 0.59, \quad C_3 \approx 0.79, \quad C_4 \approx 0.77, \quad C_5 \approx 0.88.$$

In order (from largest to smallest), the ranking is

$$v_5 > v_3 > v_4 > v_2 > v_1.$$

(c) Case 2: $\alpha = 0$, $\beta = 1$

Now

$$\mathbf{C}_p = \beta (I - \alpha M)^{-1} \mathbf{1} = 1 \cdot (I - 0 \cdot M)^{-1} \mathbf{1} = I^{-1} \mathbf{1} = \mathbf{1}.$$

Hence each node has centrality 1. The centralities are all equal:

$$\mathbf{C}_p = (1, 1, 1, 1, 1).$$

2. Discuss the effects of different values of α and β for this particular problem.

Role of α . The parameter α controls how strongly the link structure (i.e. the matrix M) influences the centrality. When α is close to 0, we effectively get

$$\mathbf{C}_p \approx \beta \mathbf{1},$$

so that all nodes have nearly the same score. As α increases (but remains in a valid range so that $(I - \alpha M)$ is invertible), the adjacency relationships matter more. Vertices with many in-links (especially from already important vertices) receive higher scores.

Role of β . The parameter β is a simple overall scaling factor in

$$\mathbf{C}_p = \beta (I - \alpha M)^{-1} \mathbf{1}.$$

If we hold α fixed, increasing or decreasing β *uniformly* rescales all centralities. Thus β affects the absolute magnitudes of the centralities but does *not* affect their relative ordering.

Similarly for this problem, we can see that β does not affect the order of centralities. However, α does affect the order, as we saw in the previous part.

3. Does β have any effect on the order of centralities?

No. Because β appears as a uniform scalar multiple,

$$\mathbf{C}_p = \beta (I - \alpha M)^{-1} \mathbf{1},$$

multiplying by $\beta > 0$ does not change the relative sizes of the components. In particular, if for some β we have $C_i > C_j$, then for any other positive β' , we still have $\beta' C_i > \beta' C_j$.

Hence *no* choice of β alone can change the ranking (order) of the centralities.

4. In PageRank, what α values guarantee that centrality values do not diverge?

In the PageRank formulation the centrality vector is computed as

$$\mathbf{C}_p = \beta (I - \alpha M)^{-1} \mathbf{1},$$

where

$$M = A^\top D^{-1},$$

$\mathbf{1}$ is the all-ones vector, and β is a scaling parameter.

A common way to understand the inverse $(I - \alpha M)^{-1}$ is via the Neumann series:

$$(I - \alpha M)^{-1} = I + \alpha M + (\alpha M)^2 + (\alpha M)^3 + \dots$$

This series is analogous to the scalar geometric series:

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots,$$

which converges if and only if $|x| < 1$.

For matrices, the series converges provided that the spectral radius of αM is less than 1:

$$\rho(\alpha M) < 1.$$

Here, $\rho(\alpha M)$ denotes the largest absolute value among the eigenvalues of αM . Let λ be the largest eigenvalue of $A^\top D^{-1}$ (and under standard normalization, also of M), so that $\rho(M) = \lambda$. Then the convergence condition becomes:

$$|\alpha| \lambda < 1 \implies |\alpha| < \frac{1}{\lambda}.$$

In many PageRank applications, M is normalized such that $\lambda = 1$. Also in Undirected graphs, the value of M is equal to one as we will prove in the next question. In these cases, the condition simplifies to:

$$0 \leq \alpha < 1.$$

A typical value used in practice is $\alpha = 0.85$.

Why is convergence of the series important?

The Neumann series

$$I + \alpha M + (\alpha M)^2 + (\alpha M)^3 + \dots$$

provides a way to compute the inverse $(I - \alpha M)^{-1}$. If this series converges, then the inverse exists in a meaningful sense and the partial sums of the series approach a finite limit. This ensures that the computed PageRank centrality vector $\mathbf{C}_p$ will have finite, well-defined values.

On the other hand, if the series does not converge (i.e., if $\rho(\alpha M) \geq 1$), then the partial sums will either grow without bound or fail to settle to any fixed number. In that case, the inverse $(I - \alpha M)^{-1}$ cannot be represented by the series, and the resulting PageRank centrality values would diverge (or be undefined), meaning they would not be finite.

5. Prove that the largest eigenvalue of $A D^{-1}$ is 1.

Recall that the matrix A is the adjacency matrix of a directed graph with n vertices and that D is the diagonal matrix whose j th diagonal entry is the outdegree d_j^{out} of vertex v_j . Thus, the (i, j) th entry of the matrix $A D^{-1}$ is given by

$$(A D^{-1})_{i,j} = A_{i,j} \cdot \frac{1}{d_j^{\text{out}}}.$$

Define the vector $\mathbf{d} = (d_1^{\text{out}}, d_2^{\text{out}}, \dots, d_n^{\text{out}})^{\top}$ and let $\mathbf{1}$ denote the all-ones vector $(1, 1, \dots, 1)^{\top}$. Notice that for each vertex v_i , the sum of the entries in row i of the adjacency matrix A is exactly d_i^{out} . That is,

$$\sum_{j=1}^n A_{i,j} = d_i^{\text{out}}.$$

Now consider the product

$$(A D^{-1}) \mathbf{d}.$$

The i th component of this product is

$$[(A D^{-1}) \mathbf{d}]_i = \sum_{j=1}^n \left(A_{i,j} \cdot \frac{1}{d_j^{\text{out}}} \right) d_j^{\text{out}} = \sum_{j=1}^n A_{i,j} = d_i^{\text{out}}.$$

Thus, we have shown that

$$(A D^{-1}) \mathbf{d} = \mathbf{d}.$$

This equation is of the form $A \mathbf{d} = \lambda \mathbf{d}$, where $\lambda = 1$.

In other words, $\mathbf{d}$ is an eigenvector of $A D^{-1}$ corresponding to the eigenvalue 1.

Furthermore, since $A D^{-1}$ is a nonnegative matrix (all entries are nonnegative) and under the usual assumptions on the graph (e.g., strong connectivity or irreducibility of the matrix), the Perron–Frobenius theorem guarantees that the spectral radius (the largest absolute value among the eigenvalues) of $A D^{-1}$ is equal to the eigenvalue associated with the positive eigenvector. Therefore, we conclude that

$$\rho(A D^{-1}) = 1,$$

i.e., the largest eigenvalue of $A D^{-1}$ is indeed 1.

4 Problem 4: Graph Analysis (25 pts)

1. Import the Les Miserables graph (the coappearance network of characters in the novel Les Miserables) and visualize the graph. (3 pts)
2. Node centrality:
 - Calculate the degree centrality, eigenvector centrality, Katz centrality, PageRank centrality, betweenness centrality, and closeness centrality for the nodes in the graph.
 - List the top 5 nodes and their corresponding centrality scores for each centrality measure. (7 pts)
3. Graph measures:
 - Calculate and report the degree distribution, global clustering coefficient, and diameter for the graph. (5 pts)
4. Implement PageRank centrality with the power method.
 - Start the power method with uniform importance scores.
 - Report the iteration step needed to converge (i.e., report t such that $\|c_t - c_{t-1}\|_2 \leq 0.01$). (10 pts)

Solution

Attached Later.

References

- [1] CS6501-016 Class Slides.
- [2] <https://en.wikipedia.org/wiki/PageRank>
- [3] Aric A. Hagberg, Daniel A. Schult and Pieter J. Swart, “Exploring network structure, dynamics, and function using NetworkX”, in Proceedings of the 7th Python in Science Conference (SciPy2008), Gael Varoquaux, Travis Vaught, and Jarrod Millman (Eds), (Pasadena, CA USA), pp. 11–15, Aug 2008

- [4] E. Angriman, A. van der Grinten, M. Hamann, H. Meyerhenke and M. Penschuck: Algorithms for Large-Scale Network Analysis and the NetworKit Toolkit. Algorithms for Big Data: DFG Priority Program 1736. Springer Nature Switzerland, 2023. pp. 3-20.

Acknowledgment

This Homework Makes use of Generative AI models such as GPT 4o, o1 and o3 for to refine the answers and make them better. However all the answers are original and are not plagiarized from any source. I acknowledge that i have not given or received unauthorized assistance on this homework.